

ETAPA 4

Definir arquitetura de pastas

Flutter - Código organizado por módulos

Documento de referência da arquitetura inicial.
Conteúdo: itens 1, 2 e 3 da Etapa 4.

1. Estrutura que vamos utilizar

Dentro da pasta lib, o projeto será organizado por módulos. Esta será a referência principal durante o desenvolvimento.

Estrutura base e módulo de autenticação

```
lib/  
|  
+-- main.dart  
|  
+-- app/  
|   +-- app.dart  
|   +-- routes/  
|   +-- theme/  
|  
+-- core/  
|   +-- constants/  
|   +-- errors/  
|   +-- services/  
|   +-- utils/  
|   +-- widgets/  
|  
+-- features/  
|  
+-- auth/  
|   +-- data/  
|       |   +-- models/  
|       |   +-- repositories/  
|       |   +-- services/  
|       |  
|       +-- presentation/  
|           +-- pages/  
|           +-- viewmodels/  
|           +-- widgets/
```

O módulo auth concentra cadastro, login, senha, validações e autenticação em duas etapas.

1. Estrutura que vamos utilizar - continuação

Os demais módulos funcionais ficam no mesmo nível dentro de features/.

```
features/  
|  
+-- profile/  
|   +-- data/  
|   +-- presentation/  
|  
+-- services/  
|   +-- data/  
|   +-- presentation/  
|  
+-- scheduling/  
|   +-- data/  
|   +-- presentation/  
|  
+-- appointments/  
|   +-- data/  
|   +-- presentation/  
|  
+-- payments/  
|   +-- data/  
|   +-- presentation/  
|  
+-- notifications/  
|   +-- data/  
|   +-- presentation/  
|  
+-- admin/  
    +-- data/  
    +-- presentation/
```

IMPORTANTE: esta organização evita concentrar todo o código em uma única pasta e facilita manutenção, testes e evolução do aplicativo.

2. O que significa cada pasta

Pasta / Arquivo	Finalidade
main.dart	Ponto de entrada do aplicativo. É o arquivo usado para iniciar o app.
app/	Configurações gerais da aplicação.
app/app.dart	Configuração principal do aplicativo, como MaterialApp e inicialização da interface.
app/routes/	Define a navegação entre as telas do aplicativo.
app/theme/	Centraliza cores, fontes, estilos e aparência visual.
core/	Reúne recursos compartilhados que podem ser utilizados por vários módulos.
core/constants/	Constantes gerais usadas pelo aplicativo.
core/errors/	Tratamento e padronização de erros.
core/services/	Serviços compartilhados entre diferentes partes do sistema.
core/utils/	Funções auxiliares e utilitários gerais.
core/widgets/	Componentes visuais reutilizáveis em várias telas.
features/	Contém os módulos funcionais: login, perfil, agenda, pagamentos, administração etc.

Visão simplificada

```
main.dart
|
+-- app/  -> configuração geral do aplicativo
|
+-- core/  -> recursos compartilhados
|
+-- features/ -> funcionalidades do sistema
```

3. Módulos do nosso aplicativo

Cada funcionalidade principal terá seu próprio módulo dentro de features/. Isso permite desenvolver e manter cada parte do sistema de forma independente e organizada.

Módulo	Responsabilidade
auth	Cadastro, login, senha, CPF, e-mail, telefone e autenticação em duas etapas.
profile	Dados do usuário e gerenciamento do perfil.
services	Serviços oferecidos pela barbearia, como nome, valor e duração.
scheduling	Calendário, dias disponíveis, horários, disponibilidade e regras de agenda.
appointments	Agendamentos, detalhes, histórico e reagendamentos.
payments	Pix, cartão de crédito, cartão de débito e status dos pagamentos.
notifications	SMS, confirmações e demais notificações relacionadas ao atendimento.
admin	Agenda administrativa, clientes, pagamentos e controles do sistema.

Exemplo: módulo de autenticação

```
features/  
+-- auth/  
  +-- data/  
    | +-- models/  
    | +-- repositories/  
    | +-- services/  
    |  
  +-- presentation/  
    +-- pages/  
    +-- viewmodels/  
    +-- widgets/
```

Neste módulo ficarão, futuramente:

- Cadastro do usuário
- Login
- Recuperação de senha
- Validação de CPF, e-mail e telefone
- Autenticação em duas etapas

REGRA IMPORTANTE: novas funcionalidades devem ser criadas dentro do módulo correspondente em features/, evitando arquivos espalhados pela pasta lib/.